



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Parikshit Task -1

Git and Github

BY

VIDYA NAYAK

251090050032

Index

Sl. No.	Contents	Page No.
1	Basic Windows Commands	3
2	Versioning	4
3	Version Control System and types	6
4	Introduction to Git and Git commands	8
5	Github account setup and creating repository	14

Basic Windows Commands

1. `mkdir foldername` – to create a new folder
2. `ls` – to list all files/directories
3. `clear` – to clear the screen
4. `cd foldername` – to open a folder listed in `ls`
5. `cd ..` – to come out of a folder
6. `cat filename.extension` – to read a file
7. `echo "Hello I am Vidya" > vidya.txt` – create a file
8. `notepad filename.txt` – to edit the contents of a file

Versioning

Versioning is a process of tracking changes to a product over time.

Each time you make changes, Git saves a new version.

Real Example

If you are making a hotel prediction website:

1. Initially only backend – **Version 1**
2. Login page added – **Version 2**
3. Payment option added – **Version 3**

Git tracks all these versions.

Why Versioning is Important?

1. Undo Changes

If your new code breaks, versioning helps to recover the older version.

This helps us to experiment freely.

2. Track Progress

Git helps us track:

- Who made the change
- What changes were made
- When the changes were made

3. Team Collaboration

Different people can work on:

1. Frontend
2. Backend
3. Login Page

without disturbing each other.

4. Professional Project Management

All software companies use version control.

Challenges Before Versioning

1. Tracking changes becomes very difficult.
2. Recovering old versions is not easy.
3. Collaboration becomes confusing.

Version Control System

A Version Control System is a tool that helps you track changes to documents, code, and files over time.

A Version Control System helps to:

- Track changes
- Save project history
- Collaborate safely

A VCS records changes over time.

Example Without VCS

- parikshit_project_latest
- parikshit_project_final
- parikshit_conclusion

This becomes **very confusing** because multiple copies of files are created.

With VCS

Version 1

↓

Version 2

↓

Version 3

All changes are tracked properly.

Benefits of VCS

1. Experiment freely
2. Disaster recovery
3. Parallel development
4. Accountability
5. History of changes

Problems Before VCS

1. Error-prone
2. Conflicts
3. Slow development

Introduction to GIT

Git is a distributed Version Control System (VCS) used to track changes in files and source code over time. It helps developers manage projects efficiently, maintain a complete history of changes, and collaborate with others without overwriting each other's work.

Git was created by Linus Torvalds in 2005 for the development of the Linux kernel. Today, it is the most widely used version control system in software development.

GIT Commands

Installing Git on Computer

1. Initialize a Repository

```
git init
```

Creates a new Git repository in a folder.

2. View Branches

```
git branch
```

Shows the branches present in the Git repository.

3. Check Repository Status

```
git status
```

Shows the branch and commit history/status.

4. Add File to Staging Area

```
git add filename.txt
```

Moves the file to the staging area.

5. Commit Changes

```
git commit -m "Added"
```

Permanently creates a snapshot of the project.

6. Switch Branch


```
git checkout feature1
```

Switches to the feature1 branch.

7. Create a New Branch

```
git branch feature1
```

Creates a new branch called feature1.

8. View Commit History

```
git log --oneline
```

Shows the commit history in a compact form.

9. Configure Username and Email

```
git config --global user.name "Vidya"
```

```
git config --global user.email "vidya@gmail.com"
```

Used to set up your Git username and email.

10. Remove a File

```
git rm filename.txt
```

Removes the file from the folder and Git tracking.

Understanding git diff

Step 1

```
cd git-parikshit
```

Step 2

```
echo "Hello I am Vidya" > hello.txt
```

Step 3

```
git add hello.txt
```

```
git commit -m "Added Hello"
```

Step 4

```
notepad hello.txt
```

Make some changes and save.

Step 5

`git diff`

`git diff` shows the changes made in a file before committing them.

12. Rename a File

`git mv oldname.txt newname.txt`

Used to rename a file.

13. Store All Text Files Inside a Folder

`mkdir textfolder`

`mv *.txt textfolder`

`mv` can also be used to move files.

14. Create a Tag

`git tag tagname`

Creates a new tag.

15. List All Tags

`git tag`

Lists all tags.

Pushing a Local Repository to GitHub

Step 1

Create a repository named `git-parikshit` on GitHub.

Step 2

`git remote add origin <repository-url>`

`git push -u origin main`

Pushes the local repository to GitHub.

Pulling Changes

`git pull origin main`

Downloads and merges changes from GitHub to the local repository.

Branching and Merging

Fast Forward Merge

When:

- Main branch is stable
- Feature branch moves forward

Step 1

`git branch feature1`

Create a new branch.

Step 2

`git checkout feature1`

Switch to the branch.

Step 3

`echo "Vidya Nayak" > vidya.txt`

`git add vidya.txt`

`git commit -m "Added Hello"`

Step 4

`git checkout main`

Step 5

`git merge feature1`

Three-Way Merge

Both branches have moved forward.

Step 1

`git checkout feature1`

Step 2

```
echo "This is feature1" > feature.txt
git add feature.txt
git commit -m "Added feature"
```

Step 3

```
git checkout main
echo "Vidya Nayak" > intro.txt
git add intro.txt
git commit -m "Intro"
```

Step 4

```
git merge feature1
```

This may create a merge commit (or merge conflicts if both branches modify the same lines).

Understanding .gitignore

Step 1

```
mkdir temp
```

Step 2

```
cd temp
echo "Do not track it" > temp.txt
```

Step 3

```
notepad .gitignore
```

Inside .gitignore:

```
temp/
```

This tells Git not to track the temp folder.

Restore

git restore --staged unstages a staged file.

Step 1

```
cd parikshit-git
```

Step 2

```
echo "Hi I am Vidya" > i.txt  
git add i.txt
```

Step 3

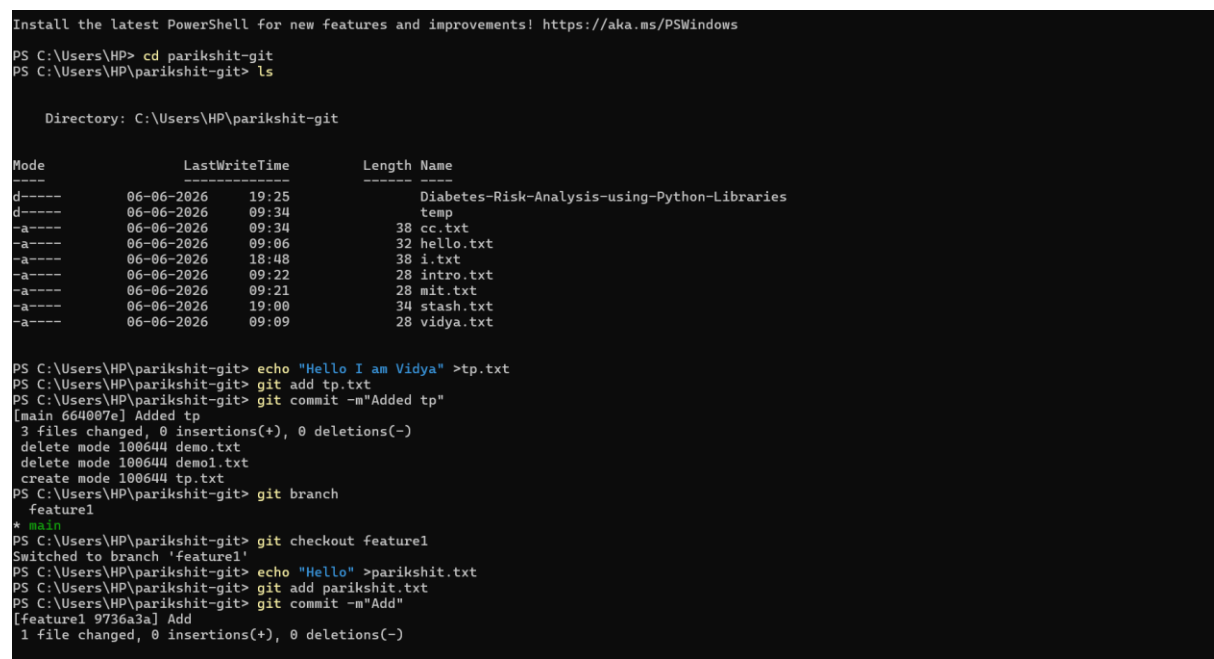
```
git restore --staged i.txt
```

Help Command

Displays documentation and help.

```
git config --help
```

Opens the help page for the git config command



```
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows  
PS C:\Users\HP> cd parikshit-git  
PS C:\Users\HP\parikshit-git> ls  
  
Directory: C:\Users\HP\parikshit-git  
  
Mode                LastWriteTime         Length Name  
----                -  
d-----          06-06-2026   19:25             Diabetes-Risk-Analysis-using-Python-Libraries  
d-----          06-06-2026   09:34             temp  
-a-----          06-06-2026   09:34             38 cc.txt  
-a-----          06-06-2026   09:06             32 hello.txt  
-a-----          06-06-2026   18:48             38 i.txt  
-a-----          06-06-2026   09:22             28 intro.txt  
-a-----          06-06-2026   09:21             28 mit.txt  
-a-----          06-06-2026   19:00             34 stash.txt  
-a-----          06-06-2026   09:09             28 vidya.txt  
  
PS C:\Users\HP\parikshit-git> echo "Hello I am Vidya" >tp.txt  
PS C:\Users\HP\parikshit-git> git add tp.txt  
PS C:\Users\HP\parikshit-git> git commit -m"Added tp"  
[main 664007e] Added tp  
3 files changed, 0 insertions(+), 0 deletions(-)  
delete mode 100644 demo.txt  
delete mode 100644 demol.txt  
create mode 100644 tp.txt  
PS C:\Users\HP\parikshit-git> git branch  
feature1  
* main  
PS C:\Users\HP\parikshit-git> git checkout feature1  
Switched to branch 'feature1'  
PS C:\Users\HP\parikshit-git> echo "Hello" >parikshit.txt  
PS C:\Users\HP\parikshit-git> git add parikshit.txt  
PS C:\Users\HP\parikshit-git> git commit -m"Add"  
[feature1 9736a3a] Add  
1 file changed, 0 insertions(+), 0 deletions(-)
```

Figure 1:Image of demonstrating git commands

```

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\HP\parikshit-git> git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)
PS C:\Users\HP\parikshit-git> git merge feature1
warning: Cannot merge binary files: cc.txt (HEAD vs. feature1)
Auto-merging cc.txt
CONFLICT (add/add): Merge conflict in cc.txt
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\HP\parikshit-git> ls

Directory: C:\Users\HP\parikshit-git


Mode                LastWriteTime         Length Name
----                -
d-----          06-06-2026   19:25             Diabetes-Risk-Analysis-using-Python-Libraries
d-----          06-06-2026   09:34                temp
-a-----          06-06-2026   20:32             38 cc.txt
-a-----          06-06-2026   20:33             38 feature.txt
-a-----          06-06-2026   09:06             32 hello.txt
-a-----          06-06-2026   20:32             38 i.txt
-a-----          06-06-2026   20:32             28 intro.txt
-a-----          06-06-2026   20:32             28 mit.txt
-a-----          06-06-2026   20:33             16 parikshit.txt
-a-----          06-06-2026   19:00             34 stash.txt
-a-----          06-06-2026   20:32             38 tp.txt
-a-----          06-06-2026   09:09             28 vidya.txt

PS C:\Users\HP\parikshit-git>

```

Figure 2: Image of demonstrating git commands

GitHub Account Setup and Creating a Repository

Part 1: Create a GitHub Account

Step 1

Open your browser and go to:

[GitHub Official Website](https://github.com)

Step 2

Click **Sign Up**.

Step 3

Enter:

- Email address
- Password
- Username

Step 4

Complete the verification process.

Step 5

Click **Create Account**.

Step 6

Verify your email using the verification link sent to your inbox.

Your GitHub account is now ready.

Part 2: Create a New Repository

Step 1

Log in to GitHub.

Step 2

Click the "+" icon in the top-right corner.

Step 3

Select **New Repository**.

Step 4

Enter the repository name.

Step 5

Choose repository visibility:

- Public
- Private

Step 6

(Optional) Select:

- Add README file
- Add .gitignore
- Choose License

Step 7

Click **Create Repository**.

The repository is now created on GitHub.

MY Repository Link

[VidyaNayak02/Parikshit-taskphase](https://github.com/VidyaNayak02/Parikshit-taskphase)